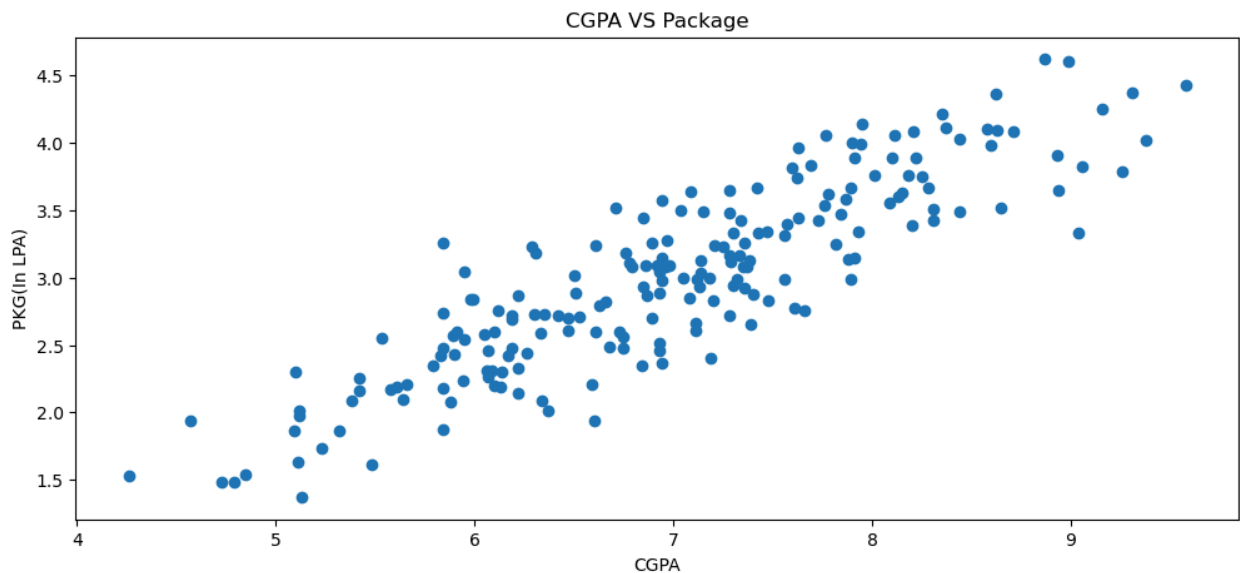


```
In [1]: # importing major libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: df=pd.read_csv('placement.csv')
```

```
In [3]: plt.figure(figsize=(12,5))
plt.scatter(df.cgpa,df.package)
plt.title('CGPA VS Package')
plt.xlabel('CGPA')
plt.ylabel('PKG(In LPA)')
plt.show()
```



```
In [4]: df.corr()
```

```
Out[4]:
```

	cgpa	package
cgpa	1.000000	0.880692
package	0.880692	1.000000

```
In [5]: # x --> independent variable // input
# y --> dependent variable // output

x= df.iloc[:,0:1]
y= df.package # 1-d/Series
x
```

Out[5]:

	cgpa
0	6.89
1	5.12
2	7.82
3	7.42
4	6.94
...	...
195	6.93
196	5.89
197	7.21
198	7.63
199	6.22

200 rows × 1 columns

Train-Test Split Workflow

Training Phase

- X_{train} → Input features for training
- y_{train} → Target values for training
- `fit()` → Model learns from X_{train} and y_{train}

Testing Phase

- X_{test} → Unseen input data
- `predict()` → Model generates predictions

Evaluation / Comparison

- Predictions → Output predicted by the model
- y_{test} → Actual (original) output
- Compare predictions with y_{test} using evaluation metrics

Flow

$X_{\text{train}} + y_{\text{train}} \rightarrow \text{fit()} \rightarrow \text{trained model}$

$X_{\text{test}} \rightarrow \text{predict()} \rightarrow \text{predictions}$

predictions ↔ y_test → evaluation

```
In [6]: #ml --> scikit learn
        # model_selection --> train_test_split
```

```
In [7]: from sklearn.model_selection import train_test_split
```

```
In [9]: x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_stat
```

```
In [10]: #x_train,y_train
         print('rows for training',x_train.shape[0])
         print('rows for testing',x_test.shape[0])
```

rows for training 160
rows for testing 40

```
In [12]: #algorithm import
         #hero
         #linear regression

         from sklearn.linear_model import LinearRegression
```

```
In [13]: lr = LinearRegression()
         lr
```

```
Out[13]: ▼ LinearRegression ⓘ ?
         LinearRegression()
```

```
In [14]: # training
         # .fit --> method
         lr.fit(x_train,y_train)
```

```
Out[14]: ▼ LinearRegression ⓘ ?
         LinearRegression()
```

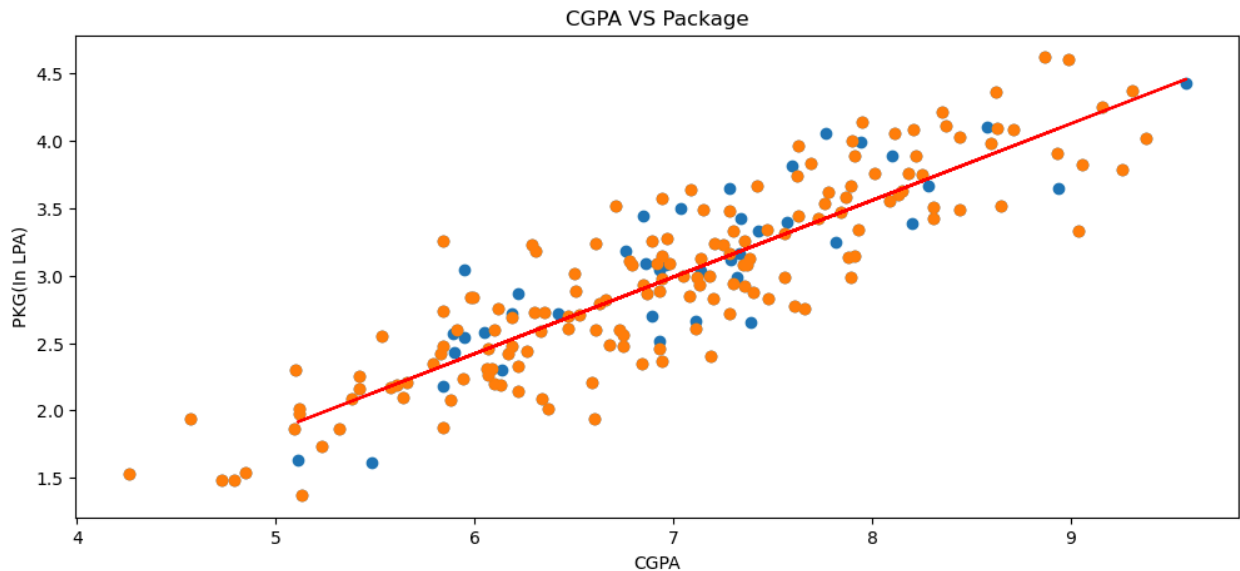
```
In [16]: #questioning
         #testing

         y_pred = lr.predict(x_test)
```

```
In [17]: # actual result
         # y_test
         y_test.iloc[0],y_pred[0]
         y_test.iloc[1],y_pred[1]
```

```
Out[17]: (np.float64(4.06), np.float64(3.4270817591718994))
```

```
In [18]: #Best fit line
plt.figure(figsize=(12,5))
plt.scatter(df.cgpa,df.package,label='test')
plt.scatter(x_train,y_train,label='train')
plt.plot(x_test,lr.predict(x_test),color='r',label='Best fit line')
plt.title('CGPA VS Package')
plt.xlabel('CGPA')
plt.ylabel('PKG(In LPA)')
plt.show()
```



```
In [19]: y_test.iloc[0],y_pred[0]
y_test.iloc[1],y_pred[1]
#y_test-y_pred
```

```
Out[19]: (np.float64(4.06), np.float64(3.4270817591718994))
```

```
In [20]: #metrics
#regression --> r2_score,mae,mse,rmse
```

```
In [21]: #r2_score/coefficient of determination/goodness of fit
#overall performance of model
```

```
In [22]: from sklearn.metrics import r2_score
r2_score(y_test,y_pred)
```

```
Out[22]: 0.7368288711522916
```

```
In [23]: # In other words , that CGPA tells 73% of variance in PKG (IN LPA)
```

```
In [ ]:
```